Assignment: Real-World Scenario to Programming Task

Group Size: 3 members per group

Submission Deadline: June 10th

1. Objective

Observe a real-world scenario, extract an algorithmic problem, and design a complete Online Judge (OJ) style programming task. This evaluates your ability to formalize problems, design test data, and implement optimal solutions.

2. Task Requirements

As a group of 3, choose a real-world situation solvable using Data Structures and Algorithms and author an original programming problem. A complete submission must include:

- A meaningful real-world context.
- A clear algorithmic problem with strict Input/Output formats.
- Reasonable constraints (e.g., variable limits, execution time).
- Sample test cases with brief explanations.
- A complete, optimal solution written in Java.
- A comprehensive suite of official test cases.

3. Submission Guidelines

Submit one `.zip` file named `StudentID_Name_Assignment.zip` with the following exact structure:

```
StudentID_Name_Assignment.zip
│
├── problem.md
├── analysis.md
├── solution.java
└── testcases/
    ├── 1.in
    ├── 1.out
    └── ...
```

3.1. problem.md (Specification)

The public-facing problem statement. It must be unambiguous so peers can solve it without asking questions. Required sections:

- `# Problem Title`
- `## Problem Statement`
- `## Input Format`
- `## Output Format`

- `## Constraints`
- `## Sample Test Cases`

3.2. analysis.md (Behind-the-Scenes)

Justifies your design choices. Required sections:

- `# Problem Analysis and Design`
- `## Real-World Context`
- `## Problem Modeling`
- `## Expected Solution` (including Time/Space Complexity)
- `## Test Case Design`
- `## Edge Cases`

3.3. testcases/ (Judging Data)

Submit 20 to 40 individual test cases (`k.in` and `k.out`). They must strategically cover:

- Basic and typical cases.
- Boundary conditions (maximum/minimum limits).
- Edge cases (zeros, empty sets).
- Large stress tests to verify performance.

3.4. solution.java (Implementation)

Your optimal Java code. It must read strictly from standard input, write to standard output, match your `.out` files exactly, and be cleanly formatted.

3.5. Automated Pre-Verification (Important Note)

Your submitted `.zip` file will be pre-verified by an automated script. You **must strictly adhere** to the folder structure, file naming conventions, and formats specified above. Any deviations will cause the script to fail, and your submission will be rejected.

4. In-Class Presentation

Prepare a brief presentation covering your chosen scenario, the algorithmic model, core solution logic, and key edge cases. Be prepared to defend your design choices and time complexity during Q&A.

5. Grading Criteria

Criterion	Description
Originality	Scenario is engaging and conceptually unique (not copied).
Clarity	`problem.md` is flawlessly written and logically sound.
Algorithm	Solution aligns with course concepts and handles constraints.
Test Rigor	Test suite thoroughly evaluates normal, edge, and stress cases.
Code Quality	`solution.java` is correct, efficient, and outputs match exactly.

Criterion	Description
Presentation	Clear articulation of the design process and confident Q&A.

Peer Review Bonus: Earn **+10/100 points** for each valid logic error, bug, or test case flaw you successfully identify in another group's submission.

6. Policies

AI tools are permitted for brainstorming or debugging, but you are fully accountable for the final product. Flawed statements, weak tests, or broken code will incur heavy deductions. Exceptional submissions may be featured in future course exams!